

「教書」RISC-V Debug 協議 ¶

背景 ¶

之前用過一些 RISC-V 核心，但是遇到調試相關的內容的時候就兩眼一抹黑，不知道原理，出了問題也不知道如何排查，趁此機會研究一下工作原理。

架構 ¶

為了調試 RISC-V 核心，需要很多部件一起工作。按 RISC-V Debug Spec 所述，有這麼幾部分：

1. 調試器：GDB，連接到 OpenOCD 啟動的 GDB 伺服器
2. Debug Translator: OpenOCD，提供 GDB Server 實現，同時會透過 FTDI 等晶片控制 JTAG
3. Debug Transport Hardware: 例如 FTDI 的晶片，可以提供 USB 接口，讓 OpenOCD 控制 JTAG 訊號 TMS/TDI/TCK 的變化，並讀取 TDO
4. Debug Transport Module: 在晶片內部的 JTAG 控制器（TAP），符合 JTAG 標準
5. Debug Module Interface: RISC-V 自訂的一系列暫存器，透過這些暫存器來控制 Debug Module 的行為
6. Debug Module: 偵錯器，控制 RISC-V 核心，同時也支援直接存取匯流排，也有內部的 Program Buffer

可以看到，DMI 是實際的調試接口，而 JTAG 可以認為是一個傳輸協定。

JTAG ¶

首先什麼是 JTAG？簡單來說，它工作流程是這樣的：

1. JTAG TAP 維護了一個狀態機，由 TMS 訊號控制
2. 當狀態機進入 CaptureDR/CaptureIR 狀態的時候，載入資料到 DR/IR 中
3. 在 ShiftDR/ShiftIR 狀態下，暫存器從 TDI 移入，從 TDO 移出

4. 當進入 UpdateDR/UpdateIR 狀態的時候，把 DR/IR 的結果輸出到其他單元

具體來說，JTAG 定義了兩類暫存器：IR 和 DR。可以把 JTAG 理解成一個小的總線，我透過 IR 選擇總線上的設備，透過 DR 向指定的設備上進行資料傳輸。例如在 RISC-V Debug Spec 裡面，規定了以下的 5 位元 IR 位址定義：

1. 0x00/0x1f：旁路
2. 0x01：ID 代碼
3. 0x10：dtmcs
4. 0x11：dmi

可以類比為有四個設備：BYPASS，IDCODE，dtmcs，dmi，對應了一些位址。如果要選擇 dtmcs 這個設備，就在 ShiftIR 階段向 TDI 輸入二進位的 00001 即可。選擇位址以後，再寫入 DR 時，操作的就是 dtmcs 設備。

那麼，每個設備是怎麼操作的呢？假如我已經透過 IR 設定了當前設備是 dtmcs，然後進入 ShiftDR 模式時，JTAG 會同時輸入和輸出。輸入的就是目前要輸入的數據，輸出的就是原來暫存器裡的結果，這個結果可能是固定的，也可能是表示上一次輸入對應的結果。

舉個例子：IDCODE 設備，在 CaptureDR 階段的時候，DR 總是會被設定為一個固定的 IDCODE，表示設備的 ID；在 Shift 的時候，這個 IDCODE 就會一位一位從 TDO 輸出，而 TDI 輸入的資料都會被忽略掉。BYPASS 裝置則是一個 1 位元的暫存器，直接從 TDI 到暫存器，暫存器到 TDO，資料就這麼流過去了。

那麼，在 RISC-V Debug 裡面，JTAG 是怎麼用的呢？我們可以這麼類比一下：CaptureDR 相當於讀取暫存器到緩衝區，然後 ShiftDR 在讀取緩衝區的同時寫入緩衝區，最後 UpdateDR 則是把緩衝區中的資料寫入到暫存器中。這和 MMIO 有點類似，只不過每次操作不是單獨的寫和讀，而是一次操作等於先讀後寫。

還是來看例子。dtmcs 這個設備表示的是 DTM 目前的狀態，它有 32 位，讀取的時候可以得到 DMI 的狀態和配置，寫入的時候可以 reset DMI。以 OpenOCD 程式碼 dtmcontrol_scan 為例子，它做了這麼多事情：

1. 首先設定 IR 為 0x10，對應 dtmcs。
2. 向 DR 寫入數據，同時讀取數據。
3. 設定 IR 為 0x11，對應 dmi，因為 dmi 操作是比較多的，所以它預設恢復到 dmi。

如果我只想讀取 `dtmcs` 暫存器，那麼只要設定寫入資料為 `0` 即可，因為暫存器的設計裡考慮到，如果寫入全 `0` 是沒有副作用的。同理，如果只想寫入 `dtmcs` 暫存器，直接寫入即可，因為設計的時候也保證讀入暫存器的值是沒有副作用的。這樣，就在一個一讀一寫的操作中，實現了讀或寫的功能。

那麼，`dmi` 暫存器的用途是什麼呢？我們前面提到過，`JTAG` 其實是一個傳輸層，而 `DMI` 又定義了一系列的暫存器，這會讓人有點混亂，為啥到處都是暫存器？又是 `JTAG` 的 `IR/DR`，又是 `dmi`，`dmi` 又有一堆寄存器，這是什麼關係？

首先我們來看 `dmi` 暫存器的定義。它由三部分組成：位址、資料和操作。由於 `JTAG` 每次操作都是一讀一寫，雖然暫存器定義差不多，但是讀寫的意思是不同的。

例如讀的時候，它表示的是上一次 `dmi` 請求的結果。位址還是上次請求的位址，資料則是上一次請求的結果，操作欄位 `0` 表示成功，`2` 表示失敗，`3` 表示還沒執行完。而寫的時候，位址和數據表示了對哪個暫存器寫入什麼數據，操作欄位 `0` 表示無操作，`1` 表示讀，`2` 表示寫。

可以看到，如果想操作 `dmi` 定義的暫存器，需要以下幾個步驟，這也是 `OpenOCD dmi_op_timeout` 要做的事：

1. 設定 `IR` 為 `0x11`，對應 `DMI`。
2. 向 `DR` 寫入請求的位址+ 資料+ 操作，丟棄讀取的結果。
3. 等待若干個週期。
4. 向 `DR` 寫入全 `0`，對應無操作，同時讀取結果，這個結果就對應上面的請求。

可以預期，如果先寫入了一個寫入操作，那麼第二次 `DR scan` 得到的結果就是是否成功寫入；如果先寫入了一個讀取操作，那麼第二次 `DR scan` 得到的結果就是目標暫存器的值。

可能看起來還是很繞，確實很繞，因為這裡有一個封裝的過程。首先，`DMI` 本身定義了一些暫存器，這些暫存器讀/寫都有一定的意義，例如控制某一個 `RISC-V` 核心暫停等等。接著，`JTAG` 需要傳輸 `DMI` 的讀取和寫入操作，同時也要考慮讀寫尚未完成的情況，該怎麼辦？結論就是透過 `DR` 來實現，寫入 `DR` 時，按照 `DR` 中的操作數，對應到 `DMI` 的寫入/讀取；然後讀取 `DR` 的時候，按照 `DMI` 的狀態，告訴 `OpenOCD` 目前是否已經完成了上一次 `DMI` 操作，和操作的結果。

直接管理介面 9

講完 JTAG 以後，終於來到 DMI 了。其實 DMI 就是一系列的暫存器，類似 MMIO 設備，只不過存取方式不是我們通常的記憶體讀寫，而是透過 JTAG 的方式進行。它有許多個暫存器，摘錄如下：

1. `dmcontrol 0x10`：調試模組控制
2. `dmstatus 0x11`：除錯模組狀態
3. `hartinfo 0x12`：Hart 信息
4. `hartsum 0x13`：Hart 摘要
5. 指令 `0x16`：抽象控制與狀態
6. `data0 0x04`：抽象資料 0
7. `progbuf0 0x20`：程式緩衝區 0
8. `sbcsc 0x38`：系統匯流排存取控制和狀態
9. `sbaddress0 0x39`：系統匯流排位址 31:0
10. `sbdatab 0x3c`：系統匯流排資料 31:0

OpenOCD 的 `examine` 函數對 DMI 初始化並進行一些參數的取得。它的操作如下：

1. 呼叫 `dtmcontrol_scan`，讀取 JTAG 裡的 `dtmcs`，可以得到 JTAG-DMI 的設定訊息
2. 向 `dmcontrol` 寫入，進行重設
3. 向 `dmcontrol` 寫入，啟用調試模組
4. 從 `hartinfo` 讀取 hart 訊息
5. 檢查各個 hart 的狀態

類似地，其他各種調試操作都是對這些 DMI 暫存器的讀取和寫入進行。RISC-V Debug Spec 附錄裡也提到如何實現偵錯器的一些功能。

例如要讀取 CPU 的暫存器（例如通用暫存器，CSR 等等）的話，有如下的方式：

第一種是 **Abstract Command**，直接向 DMI 寫入要暫存器編號，就可以實作讀取/寫入。

第二種是 **Program Buffer**。它是一塊小的程式碼存儲，可以透過 DMI 向其中寫入指令，例如 `csrw s0, mstatus; ebreak`，然後設定 `s0` 暫存器的值，再執行 Program Buffer 裡的程式碼。

以 OpenOCD 程式碼為例，`register_read_abstract` 做了以下操作：

1. 找到要讀取的暫存器對應的 **Abstract Register Number**
2. 進行 **transfer** 指令，DM 會讀取對應暫存器到 `data0` 中
3. 從 `data0` 讀取暫存器內容

如果要讀取記憶體的話，也有兩種方法。一種是直接向 **DMI** 寫入要讀取的匯流排位址，然後再向指定的暫存器讀取資料。第二種還是利用 **Program Buffer**，寫入一條 `lw s0, 0(s0)` 指令，然後先向 `s0` 寫入位址，執行 **Program Buffer** 後，再把 `s0` 暫存器的值讀出來。

抽象指令實現 ¶

那麼，如何實作上面提到的 **Abstract Command**（例如讀寫暫存器，讀寫記憶體等）呢？**Debug Spec** 裡面提到一種 **Execution-Based** 的方式，也就是在 **Debug mode** 下，核心依然在執行程式碼，只不過執行的是調試用的特殊程式碼。它做的就是輪詢 **Debug Module** 等待指令，接受到指令以後，就去讀寫寄存器/內存，然後透過 `data0-12` 來傳輸資料。

這裡還有一個比較特別的點，就是讀取暫存器的時候，暫存器的編號是直接記錄在指令中的，所以可以讓 **Debug Module** 動態產生指令，然後讓核心刷新 **ICache** 然後跳過去。另外，還可以利用 `dscratch0/dscratch1` 暫存器來保存 `gpr`，然後用 `dret` 退出的時候再恢復，這樣就有兩個 `gpr` 可以用來實現功能了，實際上這已經夠用了（一個技巧是，把地址設為 `0` 附近，然後直接用 `zero` 寄存器加偏移來尋址）。

單步驟調試實現 ¶

在 `dcscr` 中，有一個值 `step` 表示是否在單步驟偵錯狀態。設 `step` 為 `1` 的時候，如果不在 **debug mode** 中，只需要記錄以及執行的指令數，當執行了一條指令後，視為下一個指令發生了進入 **debug mode** 的異常，這樣就實現了單步調試。

軟體斷點實現 ¶

調試器為了打斷點，一個簡單的方式是，往斷點處寫入 `ebreak` 指令，然後設定 `dcscr` 的 `ebreakm/s/u`，表示在這些特權集裡，`ebreak` 是進入 **debug mode**，而不是原來的

處理過程。然後，程式運行到 `ebreak` 指令的時候，進入 `debug mode`，`openocd` 發現核心進入 `halted` 狀態後，讓 `gdb` 繼續進行偵錯。

硬體方面的實作方法就是，在遇到 `ebreak` 的時候，判斷一下目前的特權集，結合 `ebreakm/s/u` 判斷跳到什麼狀態。此外，由於它會寫入指令到內存，所以還需要執行 `fence.i` 指令，而 `OpenOCD` 需要依賴 `progbuf` 來執行 `fence.i` 指令，所以為了讓這個方案運作，還得實作 `Program Buffer`。

當然了，軟體斷點也有局限性，例如記憶體不可寫，例如 `ROM`，不能覆蓋裡面的指令，這樣就有可能出問題。而且硬體斷點效能也更好，不需要來回這樣寫指令。

半主機 ¶

`ARM` 有一種 `semihosting` 機制，就是處理器執行一種特定的指令序列，然後調試器看到整個序列的時候，不是進入 `GDB` 調試狀態，而是去進行一些操作，比如輸出信息，讀寫文件等等，然後結果通過 `JTAG` 寫回去。`OpenOCD` 給 `RISC-V` 也做了類似的 `semihosting` 機制，只不過觸發的指令序列不大一樣，但是機制是類似的。

如果用過 `Rocket Chip` 模擬的或以前的 `ucb-bar/fpga-zynq` 專案的話，會知道還有一個目的有些類似的東西：`HTIF + fesvr`，它是透過 `fromhost/tohost` 兩組位址來進行通信，但是這個方法缺點是需要 `poll tohost/fromhost` 位址的內容，相對來說比較麻煩。

程序緩衝區 ¶

此外，`debug spec` 還有一個可選的功能，就是 `Program Buffer`，偵錯器可以往裡面插入自訂的指令，然後結合 `abstract command` 來操作。這樣就可以做一些比較有效率的操作，像是 `OpenOCD` 實作的批次寫入記憶體：

```
sw s1, 0(a0)
addi a0, a0, 4
ebreak
```

並且設定 `abstractauto`，然後重複的操作是往 `s1` 裡面寫入新的數據，然後跳到 `program buffer`，進行上面的 `sw` 操作，這樣就可以一次 `dmi` 請求完成一次內存的寫入，比較高效。

參考文件 ¶

1. RISC-V 調試規範 0.13
 2. IEEE JTAG 標準 1149.1-2013
 3. OpenOCD 相關程式碼
- 2023 年 7 月 14 日 2021 年 12 月 12 日